FARMACIA LUFAR

Sistema de gestión, control y ventas

Modelo Entidad-Relación y Modelo Relacional

Documento de diseño de base de datos

Django + SQLite

Proyecto:	Farmacia Lufar
Base:	Actividad 1.3 — Proyecto Fase I
Alcance:	Productos, inventario, compras, ventas, clientes, proveedores, usuarios y reportes
Fecha:	Noviembre de 2026

Documento de apoyo para el diseño lógico de la base de datos.
Las decisiones aquí presentadas conservan la arquitectura modular del proyecto.

1 Alcance y convenciones

El modelo se deriva de los requerimientos funcionales del documento *Actividad 1.3 — Proyecto Fase I*. Cubre la gestión de productos, categorías, proveedores, clientes, compras, ventas, inventario, usuarios y reportes. Se conservan las tablas ya implementadas en el proyecto Django: `Cliente`, `Producto`, `Venta` y `DetalleVenta`.

Supuestos de diseño

- Se incluyen **lotes y fechas de vencimiento**, porque son parte del alcance farmacéutico y permiten aplicar FEFO (primero en vencer, primero en salir). Si no se confirman, el modelo puede simplificarse como se indica al final.
- El pago de las ventas es **solo en efectivo**. No se crea una tabla de pagos ni de facturación fiscal.
- Las ventas son **inmutables**: no se eliminan. Si se habilita una anulación, se registra un movimiento inverso de inventario.
- El stock se controla por lote. El stock total de un producto se obtiene sumando sus lotes; no se duplica en `Producto`.
- La aplicación es local, con SQLite, sin integración con hardware ni servicios externos en esta fase.

Convenciones del diagrama y del esquema

- `id`: clave primaria (subrayada).
- **`campo`**: clave foránea (negrita).
- UK: clave única.
- **1:N**, **N:M** y **0..1**: cardinalidades. **0..1** indica una relación opcional.
- En el modelo relacional, una flecha → indica la tabla referenciada por una clave foránea.

Tipos de datos recomendados

Tipo	Uso
`INTEGER`	Identificadores, cantidades y llaves.
`VARCHAR(n)`	Códigos, nombres, documentos y textos cortos.
`DECIMAL(12,2)`	Precios, costos, totales e impuestos.
`DATE`	Fechas de compra, recepción, fabricación y vencimiento.
`DATETIME`	Marcas de tiempo de creación, actualización y movimientos.
`BOOLEAN`	Indicadores de estado. SQLite los almacena como 0/1.
`TEXT`	Observaciones y rutas de archivos.

2 Modelo entidad-relación

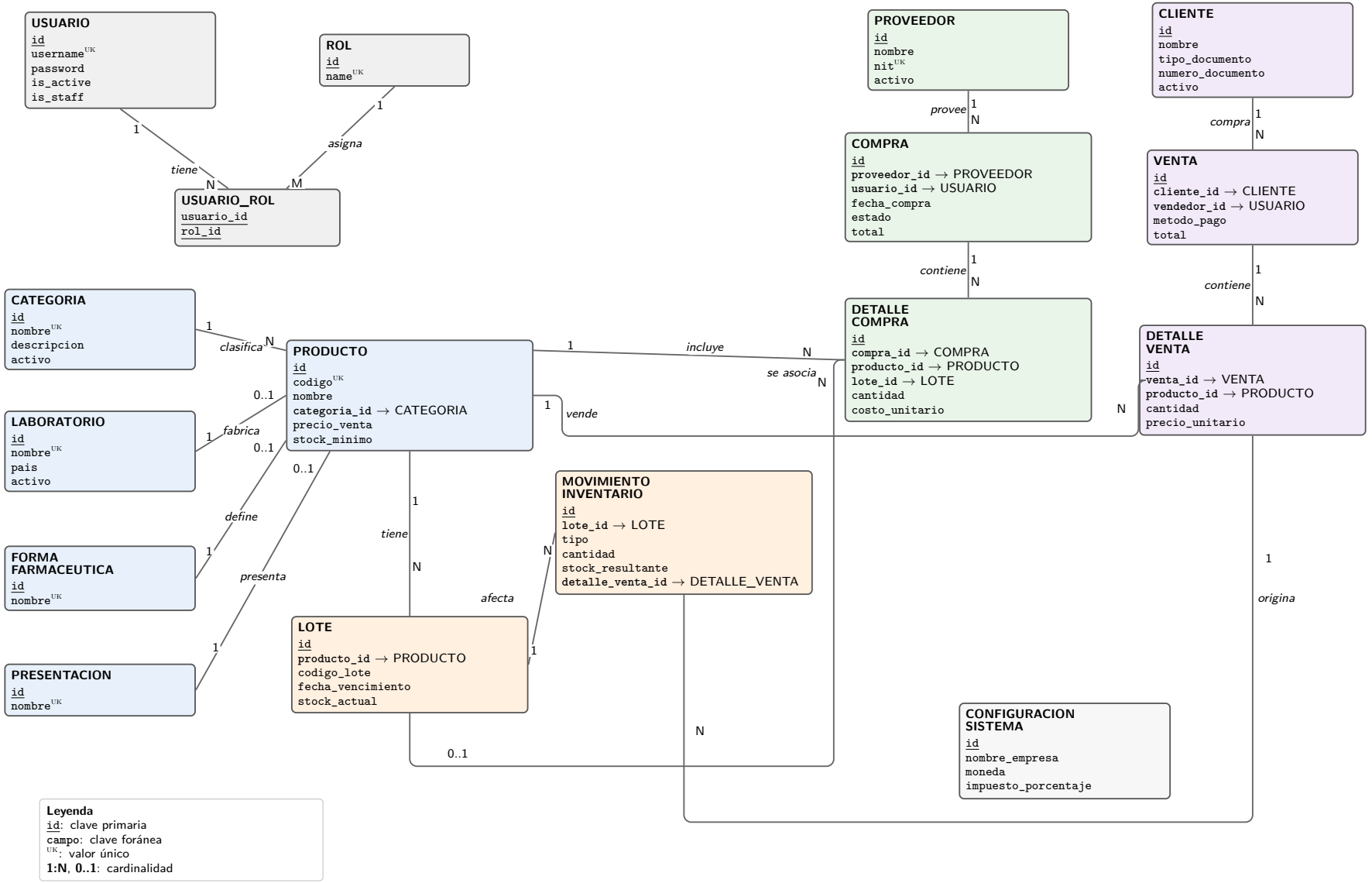


Figure 1: Modelo entidad-relación de Farmacia Lufar. Las referencias de auditoría desde `USUARIO` hacia `CLIENTE`, `COMPRA`, `VENTA` y `MOVIMIENTO_INVENTARIO` se detallan en el modelo relacional para no saturar el diagrama.

Cardinalidades principales

Relación	Cardinalidad
CATEGORIA – PRODUCTO	1:N
PRODUCTO – LOTE	1:N
LOTE – MOVIMIENTO_INVENTARIO	1:N
PROVEEDOR – COMPRA	1:N
COMPRA – DETALLE_COMPRA	1:N
LOTE – DETALLE_COMPRA	1:N (lote opcional en el detalle)
CLIENTE – VENTA	1:N (cliente opcional en la venta)
VENTA – DETALLE_VENTA	1:N
PRODUCTO – DETALLE_VENTA	1:N
DETALLE_VENTA – MOVIMIENTO_INVENTARIO	1:N
USUARIO – ROL	N:M mediante USUARIO_ROL

3 Modelo relacional

Convenciones

En las relaciones se usa id para la clave primaria, **campo** para la clave foránea y ^{UK} para los valores únicos. Las flechas → indican la relación referenciada. Los campos marcados como opcionales pueden ser NULL.

Relación	Atributos, claves y referencias
Módulo de seguridad (Django)	
USUARIO	id, username^{UK}, password, first_name, last_name, email, is_active, is_staff, is_superuser.
ROL	id, name^{UK}.
USUARIO_ROL	usuario_id → USUARIO.id, rol_id → ROL.id. Clave primaria compuesta y unicidad de la pareja.
Módulo de catálogo	
CATEGORIA	id, nombre^{UK}, descripcion, activo.
LABORATORIO	id, nombre^{UK}, pais, activo.
FORMA_FARMACEUTICA	id, nombre^{UK}.
PRESENTACION	id, nombre^{UK}.
PRODUCTO	id, codigo^{UK}, nombre, descripcion, principio_activo, **categoria_id** → CATEGORIA.id, **laboratorio_id** → LABORATORIO.id (opcional), **forma_farmaceutica_id** → FORMA_FARMACEUTICA.id (opcional), **presentacion_id** → PRESENTACION.id (opcional), concentracion, unidad_medida, precio_venta, stock_minimo, activo, creado_en, actualizado_en.

Relación	Atributos, claves y referencias
Módulo de compras	
PROVEEDOR	id, nombre, nit[UK] (opcional), telefono, email, direccion, contacto, activo, creado_en, actualizado_en.
COMPRA	id, **proveedor_id** → PROVEEDOR.id, **usuario_id** → USUARIO.id, numero_factura, fecha_compra, fecha_recepcion (opcional), estado, subtotal, impuesto, total, observaciones, creado_en, actualizado_en.
DETALLE_COMPRA	id, **compra_id** → COMPRA.id, **producto_id** → PRODUCTO.id, **lote_id** → LOTE.id (opcional hasta la recepción), cantidad, costo_unitario, subtotal.
Módulo de inventario	
LOTE	id, **producto_id** → PRODUCTO.id, codigo_lote, fecha_fabricacion (opcional), fecha_vencimiento (opcional), fecha_ingreso, stock_actual, activo. Único por (producto_id, codigo_lote).
MOVIMIENTO_INVENTARIO	id, **lote_id** → LOTE.id, tipo, cantidad, stock_resultante, **detalle_venta_id** → DETALLE_VENTA.id (opcional), motivo, **usuario_id** → USUARIO.id, fecha.
Módulo de clientes y ventas	
CLIENTE	id, nombre, tipo_documento, numero_documento, telefono, activo, **creado_por_id** → USUARIO.id (opcional), creado_en, actualizado_en. Único por (tipo_documento, numero_documento) cuando ambos no están vacíos.
VENTA	id, **cliente_id** → CLIENTE.id (opcional), **vendedor_id** → USUARIO.id, metodo_pago, monto_recibido, total, creado_en.
DETALLE_VENTA	id, **venta_id** → VENTA.id, **producto_id** → PRODUCTO.id, codigo_producto, nombre_producto, cantidad, precio_unitario, creado_en. Único por (venta_id, producto_id).
Módulo de configuración	
CONFIGURACION_SISTEMA	id, nombre_empresa, nit, direccion, telefono, email, moneda, impuesto_porcentaje, logo, stock_minimo_default, actualizado_en. Una sola fila lógica (id = 1).

Restricciones de integridad

1. LOTE.stock_actual >= 0 y MOVIMIENTO_INVENTARIO.stock_resultante >= 0. No se permite stock negativo.
2. PRODUCTO.precio_venta >= 0, PRODUCTO.stock_minimo >= 0, DETALLE_COMPRA.cantidad > 0, DETALLE_COMPRA.costo_unitario >= 0.
3. VENTA.total >= 0 y VENTA.monto_recibido >= VENTA.total. La venta se registra solo en efectivo.
4. DETALLE_VENTA.cantidad entre 1 y 9999; DETALLE_VENTA.precio_unitario >= 0.
5. DETALLE_VENTA conserva codigo_producto, nombre_producto y precio_unitario como copia histórica.
6. COMPRA.estado solo puede ser BORRADOR, RECIBIDA o ANULADA. Solo RECIBIDA genera movimientos de entrada.
7. MOVIMIENTO_INVENTARIO.tipo solo puede ser INICIAL, ENTRADA, SALIDA, AJUSTE_POSITIVO o AJUSTE_NEGATIVO.
8. CLIENTE exige que tipo_documento y numero_documento se registren juntos, o ambos vacíos.

9. LOTE es único por (producto_id, codigo_lote). Los lotes vencidos o inactivos no se usan para vender.
10. Las operaciones que afectan varias tablas (venta + inventario, recepción de compra + inventario) se ejecutan dentro de transaction.atomic().

Índices recomendados

Tabla	Índice
PRODUCTO	(codigo) único, (nombre), (categoria_id)
LOTE	(producto_id, codigo_lote) único, (producto_id, fecha_vencimiento)
COMPRA	(fecha_compra), (proveedor_id), (estado)
VENTA	(creado_en), (cliente_id), (vendedor_id)
DETALLE_VENTA	(venta_id), (producto_id)
MOVIMIENTO_INVENTARIO	(lote_id, fecha), (fecha)

Correspondencia con las aplicaciones Django

App	Tablas propuestas
accounts	USUARIO, ROL, USUARIO_ROL (tablas de Django)
catalogo	CATEGORIA, LABORATORIO, FORMA_FARMACEUTICA, PRESENTACION, PRODUCTO
compras	PROVEEDOR, COMPRA, DETALLE_COMPRA
inventario	LOTE, MOVIMIENTO_INVENTARIO
clientes	CLIENTE
ventas	VENTA, DETALLE_VENTA
core	CONFIGURACION_SISTEMA
reportes	Sin tablas propias: consultas de solo lectura

Orden de implementación

1. **Catálogo:** crear CATEGORIA y los catálogos opcionales; ampliar PRODUCTO. Para los productos existentes, crear una categoría “General” en una migración de datos.
2. **Inventario:** crear LOTE y MOVIMIENTO_INVENTARIO. El movimiento puede referenciar ventas.DETALLE_VENTA; no crear la relación inversa desde ventas hacia inventario.
3. **Compras:** crear PROVEEDOR, COMPRA y DETALLE_COMPRA; DETALLE_COMPRA.lote_id referencia inventario.LOTE.
4. **Ventas:** integrar el servicio de inventario en registrar_venta para generar salidas FEFO. No se requieren cambios estructurales en VENTA ni en DETALLE_VENTA.
5. **Configuración:** agregar CONFIGURACION_SISTEMA solo si se implementa el módulo de configuraciones.
6. **Reportes:** implementar consultas sobre ventas, compras, lotes y movimientos; no crear tablas.

Simplificación si no se confirman lotes

Si la entrevista descarta el control por lotes y vencimientos, se puede usar una versión más simple:

- Eliminar `LOTE`.
- Agregar `PRODUCTO.stock_actual` o una tabla `EXISTENCIA_PRODUCTO(producto_id, cantidad_actual, stock_minimo)`.
- Cambiar `MOVIMIENTO_INVENTARIO.lote_id` por `producto_id`.
- Quitar `DETALLE_COMPRA.lote_id`.

Se recomienda conservar `MOVIMIENTO_INVENTARIO`, porque permite auditar entradas, salidas y ajustes aunque no se usen lotes.